

```

import matplotlib.pyplot as plt
import pandas as pd

def analisar_bombas():
    print("=== SISTEMA DE ANÁLISE DE EFICIÊNCIA DE BOMBAS CENTRÍFUGAS ===")

    try:
        N = int(input("Digite o número de bombas a serem analisadas (N): "))
        if N <= 0:
            print("O número de bombas deve ser maior que zero.")
            return
    except ValueError:
        print("Entrada inválida. Digite um número inteiro.")
        return

    # Listas para armazenar os dados
    dados_bombas = []

    # Coleta de dados
    for i in range(N):
        print(f"\n--- Cadastro da Bomba {i+1} ---")
        tag = input("Identificação (TAG) ou Setor: ").strip().upper()
        vazao_nom = float(input("Vazão nominal de projeto (m³/h): "))
        potencia = float(input("Potência nominal do motor (kW): "))
        rendimento = float(input("Rendimento atual da bomba (η em %): "))

        horas_dia = float(input("Horas de operação diária (h): "))
        custo_energia = float(input("Custo da energia elétrica (R$/kWh): "))
        custo_mnt = float(
            input("Custo de manutenção preventiva por hora (R$/h): ")
        )

        # Cálculos das Variáveis
        consumo_diario = potencia * horas_dia
        custo_operacional = (consumo_diario * custo_energia) + (
            horas_dia * custo_mnt
        )
        vazao_real = vazao_nom * (rendimento / 100)

```

```

# Classificação da Criticidade
if rendimento > 85:
    criticidade = "Baixo (OK) "
elif 70 <= rendimento <= 85:
    criticidade = "Médio (Atenção) "
else:
    criticidade = "Alto (Crítico) "

# Armazenando no dicionário
dados_bombas.append(
    {
        "TAG": tag,
        "Vazão Nominal (m³/h)": vazao_nom,
        "Vazão Real (m³/h)": vazao_real,
        "Potência (kW)": potencia,
        "Rendimento (%)": rendimento,
        "Consumo Diário (kWh)": consumo_diario,
        "Custo Diário (R$)": custo_operacional,
        "Criticidade": criticidade,
    }
)

# Criação do DataFrame para manipulação de dados e exibição
df = pd.DataFrame(dados_bombas)

# --- SAÍDA 1: Tabela de Resultados ---
print("\n" + "=" * 80)
print("TABELA DE RESULTADOS TÉCNICOS E FINANCEIROS")
print("=" * 80)
print(
    df.to_string(
        index=False,
        formatters={
            "Vazão Real (m³/h)": "{:.2f}".format,
            "Consumo Diário (kWh)": "{:.2f}".format,
            "Custo Diário (R$)": "R$ {:.2f}".format,
            "Rendimento (%)": "{:.1f}%".format,
        },
    )
)
print("=" * 80)

# --- SAÍDA 5: Identificação da bomba com pior rendimento ---

```

```

pior_bomba = df.loc[df["Rendimento (%)"].idxmin()]
print(f"\n[ALERTA DE MANUTENÇÃO]")
print(
    f"A bomba mais crítica (pior rendimento) é a TAG:
{pior_bomba['TAG']} com apenas {pior_bomba['Rendimento (%)']:.1f}% de
eficiência."
)
print(
    f"Ela está operando com uma vazão real de {pior_bomba['Vazão
Real (m³/h)']:.2f} m³/h (Nominal: {pior_bomba['Vazão Nominal (m³/h)']}
m³/h)."
)

# --- GERAÇÃO DOS GRÁFICOS (SAÍDAS 2, 3 e 4) ---
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle(
    "Análise de Desempenho e Custos da Frota de Bombas",
    fontsize=16
)

# Gráfico 1: Vazão Nominal vs. Vazão Real (Barras)
x = range(len(df["TAG"]))
width = 0.35
axes[0, 0].bar(
    [i - width / 2 for i in x],
    df["Vazão Nominal (m³/h)"],
    width,
    label="Nominal",
    color="#1f77b4",
)
axes[0, 0].bar(
    [i + width / 2 for i in x],
    df["Vazão Real (m³/h)"],
    width,
    label="Real Estimada",
    color="#aec7e8",
)
axes[0, 0].set_ylabel("Vazão (m³/h)")
axes[0, 0].set_title("Comparativo: Vazão Nominal vs. Vazão Real")
axes[0, 0].set_xticks(x)
axes[0, 0].set_xticklabels(df["TAG"])
axes[0, 0].legend()
axes[0, 0].grid(axis="y", linestyle="--", alpha=0.7)

```

```

# Gráfico 2: Custo Operacional Diário (Barras)
axes[0, 1].bar(df["TAG"], df["Custo Diário (R$)"], color="#d62728")
axes[0, 1].set_ylabel("Custo Diário (R$)")
axes[0, 1].set_title("Custo Operacional Diário por Bomba")
axes[0, 1].grid(axis="y", linestyle="--", alpha=0.7)
for i, v in enumerate(df["Custo Diário (R$)"]):
    axes[0, 1].text(i, v + (v * 0.01), f"R${v:.2f}", ha="center")

# Gráfico 3: Distribuição dos Níveis de Criticidade (Pizza)
contagem_criticidade = (
    df["Criticidade"].value_counts().reindex(["Baixo (OK)", "Médio (Atenção)", "Alto (Crítico)"], fill_value=0)
)
# Filtra apenas as categorias que existem na amostra atual para evitar divisões por zero no gráfico de pizza
contagem_criticidade = contagem_criticidade[contagem_criticidade > 0]

cores = {
    "Baixo (OK)": "#2ca02c",
    "Médio (Atenção)": "#ff7f0e",
    "Alto (Crítico)": "#d62728",
}
cores_pizza = [cores[idx] for idx in contagem_criticidade.index]

axes[1, 0].pie(
    contagem_criticidade,
    labels=contagem_criticidade.index,
    autopct="%1.1f%%",
    startangle=140,
    colors=cores_pizza,
)
axes[1, 0].set_title("Distribuição dos Níveis de Criticidade")

# Espaço reservado para o Diagrama de Processo (Representação textual/Visual simulada)
axes[1, 1].axis("off")
axes[1, 1].text(
    0.5,
    0.5,
    "Fluxo de Processo Típico:\n\n[Sução / Reservatório]\n
|\n      ▼\n[Bomba Centrífuga] —(Perda de η)—> [Calor/Vibração]\n

```

```

|\n          ▼\n[Vazão Real Medida]\n          |\n          ▼\n[Destino /
Processo Industrial]",
    ha="center",
    va="center",
    fontsize=11,
    bbox=dict(boxstyle="round", facecolor="#f5f5f5",
edgecolor="#000"),
)
axes[1, 1].set_title("Esquema do Transporte de Fluidos")

plt.tight_layout()

# Salva o gráfico em arquivo e mostra na tela
nome_grafico = "analise_bombas_graficos.png"
plt.savefig(nome_grafico)
print(f"\n[INFO] Os gráficos comparativos foram salvos como
'{nome_grafico}'.")
plt.show()

```

```

# Executar o programa
if __name__ == "__main__":

```

ERRO

```

except ValueError:
    print("Entrada inválida. Digite um número inteiro.")
    return

# Listas para armazenar os dados
dados_bombas = []

# Coleta de dados
for i in range(N):
    print("\n--- Cadastro da Bomba {i+1} ---")
    print(f"\n--- Cadastro da Bomba {i+1} ---")
    tag = input("Identificação (TAG) ou Setor: ").strip().upper()
    vazao_nom = float(input("Vazão nominal de projeto (m³/h): "))
    potencia = float(input("Potência nominal do motor (kW): "))
    rendimento = float(input("Rendimento atual da bomba (η em %): "))
    horas_dia = float(input("Horas de operação diária (h): "))
    custo_energia = float(input("Custo da energia elétrica (R$/kWh): "))
    custo_mnt = float(

```

Explica este erro:

SyntaxError: unexpected character after line continuation cha

◆ O erro 'SyntaxError: unexpected character after line continuation character' está a acontecer porque o caractere de nova linha (`\n`) está mal colocado. Ele precisa de estar dentro da string que está a ser impressa. Corrigi-o movendo o `\n` para dentro da string.

👏 🗨

CÓDIGO FINAL

```
import matplotlib.pyplot as plt
import pandas as pd

def analisar_bombas():
    print("=== SISTEMA DE ANÁLISE DE EFICIÊNCIA DE BOMBAS CENTRÍFUGAS ===")

    try:
        N = int(input("Digite o número de bombas a serem analisadas (N): "))
        if N <= 0:
            print("O número de bombas deve ser maior que zero.")
            return
    except ValueError:
        print("Entrada inválida. Digite um número inteiro.")
        return

    # Listas para armazenar os dados
    dados_bombas = []

    # Coleta de dados
    for i in range(N):
        print(f"\n--- Cadastro da Bomba {i+1} ---")
        tag = input("Identificação (TAG) ou Setor: ").strip().upper()
        vazao_nom = float(input("Vazão nominal de projeto (m³/h): "))
        potencia = float(input("Potência nominal do motor (kW): "))
        rendimento = float(input("Rendimento atual da bomba (η em %): "))

        horas_dia = float(input("Horas de operação diária (h): "))
        custo_energia = float(input("Custo da energia elétrica (R$/kWh): "))
        custo_mnt = float(
            input("Custo de manutenção preventiva por hora (R$/h): ")
        )

        # Cálculos das Variáveis
        consumo_diario = potencia * horas_dia
        custo_operacional = (consumo_diario * custo_energia) + (
            horas_dia * custo_mnt
        )
        vazao_real = vazao_nom * (rendimento / 100)
```

```

# Classificação da Criticidade
if rendimento > 85:
    criticidade = "Baixo (OK)"
elif 70 <= rendimento <= 85:
    criticidade = "Médio (Atenção)"
else:
    criticidade = "Alto (Crítico)"

# Armazenando no dicionário
dados_bombas.append(
    {
        "TAG": tag,
        "Vazão Nominal (m³/h)": vazao_nom,
        "Vazão Real (m³/h)": vazao_real,
        "Potência (kW)": potencia,
        "Rendimento (%)": rendimento,
        "Consumo Diário (kWh)": consumo_diario,
        "Custo Diário (R$)": custo_operacional,
        "Criticidade": criticidade,
    }
)

# Criação do DataFrame para manipulação de dados e exibição
df = pd.DataFrame(dados_bombas)

# --- SAÍDA 1: Tabela de Resultados ---
print("\n" + "=" * 80)
print("TABELA DE RESULTADOS TÉCNICOS E FINANCEIROS")
print("=" * 80)
print(
    df.to_string(
        index=False,
        formatters={
            "Vazão Real (m³/h)": "{:.2f}".format,
            "Consumo Diário (kWh)": "{:.2f}".format,
            "Custo Diário (R$)": "R$ {:.2f}".format,
            "Rendimento (%)": "{:.1f}%".format,
        },
    )
)
print("=" * 80)

```

```

# --- SAÍDA 5: Identificação da bomba com pior rendimento ---
pior_bomba = df.loc[df["Rendimento (%)"].idxmin()]
print(f"\n[ALERTA DE MANUTENÇÃO]")
print(
    f"A bomba mais crítica (pior rendimento) é a TAG:
{pior_bomba['TAG']} com apenas {pior_bomba['Rendimento (%)']:.1f}% de
eficiência."
)
print(
    f"Ela está operando com uma vazão real de {pior_bomba['Vazão
Real (m³/h)']:.2f} m³/h (Nominal: {pior_bomba['Vazão Nominal (m³/h)']}
m³/h)."
)

# --- GERAÇÃO DOS GRÁFICOS (SAÍDAS 2, 3 e 4) ---
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle(
    "Análise de Desempenho e Custos da Frota de Bombas",
    fontsize=16
)

# Gráfico 1: Vazão Nominal vs. Vazão Real (Barras)
x = range(len(df["TAG"]))
width = 0.35
axes[0, 0].bar(
    [i - width / 2 for i in x],
    df["Vazão Nominal (m³/h)"],
    width,
    label="Nominal",
    color="#1f77b4",
)
axes[0, 0].bar(
    [i + width / 2 for i in x],
    df["Vazão Real (m³/h)"],
    width,
    label="Real Estimada",
    color="#aec7e8",
)
axes[0, 0].set_ylabel("Vazão (m³/h)")
axes[0, 0].set_title("Comparativo: Vazão Nominal vs. Vazão Real")
axes[0, 0].set_xticks(x)
axes[0, 0].set_xticklabels(df["TAG"])
axes[0, 0].legend()

```



```

axes[0, 0].grid(axis="y", linestyle="--", alpha=0.7)

# Gráfico 2: Custo Operacional Diário (Barras)
axes[0, 1].bar(df["TAG"], df["Custo Diário (R$)"], color="#d62728")
axes[0, 1].set_ylabel("Custo Diário (R$)")
axes[0, 1].set_title("Custo Operacional Diário por Bomba")
axes[0, 1].grid(axis="y", linestyle="--", alpha=0.7)
for i, v in enumerate(df["Custo Diário (R$)"]):
    axes[0, 1].text(i, v + (v * 0.01), f"R${v:.2f}", ha="center")

# Gráfico 3: Distribuição dos Níveis de Criticidade (Pizza)
contagem_criticidade = (
    df["Criticidade"].value_counts().reindex(["Baixo (OK)", "Médio (Atenção)", "Alto (Crítico)"], fill_value=0)
)

# Filtra apenas as categorias que existem na amostra atual para evitar divisões por zero no gráfico de pizza
contagem_criticidade = contagem_criticidade[contagem_criticidade > 0]

cores = {
    "Baixo (OK)": "#2ca02c",
    "Médio (Atenção)": "#ff7f0e",
    "Alto (Crítico)": "#d62728",
}

cores_pizza = [cores[idx] for idx in contagem_criticidade.index]

axes[1, 0].pie(
    contagem_criticidade,
    labels=contagem_criticidade.index,
    autopct="%1.1f%%",
    startangle=140,
    colors=cores_pizza,
)

axes[1, 0].set_title("Distribuição dos Níveis de Criticidade")

# Espaço reservado para o Diagrama de Processo (Representação textual/Visual simulada)
axes[1, 1].axis("off")
axes[1, 1].text(
    0.5,
    0.5,

```

```

        "Fluxo de Processo Típico:\n\n[Sução / Reservatório]\n
|\n      ▼\n[Bomba Centrífuga] —(Perda de η)—> [Calor/Vibração]\n
|\n      ▼\n[Vazão Real Medida]\n      |\n      ▼\n[Destino /
Processo Industrial]",
        ha="center",
        va="center",
        fontsize=11,
        bbox=dict(boxstyle="round", facecolor="#f5f5f5",
edgecolor="#000"),
    )
    axes[1, 1].set_title("Esquema do Transporte de Fluidos")

plt.tight_layout()

# Salva o gráfico em arquivo e mostra na tela
nome_grafico = "analise_bombas_graficos.png"
plt.savefig(nome_grafico)
print(f"\n[INFO] Os gráficos comparativos foram salvos como
'{nome_grafico}'.")
plt.show()

# Executar o programa
if __name__ == "__main__":
    analisar_bombas()

```

RESULTADO

=== SISTEMA DE ANÁLISE DE EFICIÊNCIA DE BOMBAS CENTRÍFUGAS ===
Digite o número de bombas a serem analisadas (N): 3

--- Cadastro da Bomba 1 ---
Identificação (TAG) ou Setor: BC-101
Vazão nominal de projeto (m³/h): 120
Potência nominal do motor (kW): 45
Rendimento atual da bomba (η em %): 88
Horas de operação diária (h): 24
Custo da energia elétrica (R\$/kWh): 0.65
Custo de manutenção preventiva por hora (R\$/h): 5.50

--- Cadastro da Bomba 2 ---
Identificação (TAG) ou Setor: BC-202A
Vazão nominal de projeto (m³/h): 80
Potência nominal do motor (kW): 30
Rendimento atual da bomba (η em %): 74
Horas de operação diária (h): 16
Custo da energia elétrica (R\$/kWh): 0.65
Custo de manutenção preventiva por hora (R\$/h): 8.00

--- Cadastro da Bomba 3 ---
Identificação (TAG) ou Setor: BC-501
Vazão nominal de projeto (m³/h): 150
Potência nominal do motor (kW): 55
Rendimento atual da bomba (η em %): 58
Horas de operação diária (h): 12
Custo da energia elétrica (R\$/kWh): 0.65
Custo de manutenção preventiva por hora (R\$/h): 15.50

TABELA DE RESULTADOS TÉCNICOS E FINANCEIROS

TAG	Vazão Nominal (m³/h)	Vazão Real (m³/h)	Potência (kW)	Rendimento (%)	Consumo Diário (kWh)	Custo Diário (R\$)	Criticidade
BC-101	120.0	105.60	45.0	88.0%	1080.00	R\$ 834.00	Baixo (OK)
BC-202A	80.0	59.20	30.0	74.0%	480.00	R\$ 440.00	Médio (Atenção)
BC-501	150.0	87.00	55.0	58.0%	660.00	R\$ 615.00	Alto (Crítico)

[ALERTA DE MANUTENÇÃO]
A bomba mais crítica (pior rendimento) é a TAG: BC-501 com apenas 58.0% de eficiência.
Ela está operando com uma vazão real de 87.00 m³/h (Nominal: 150.0 m³/h).

[INFO] Os gráficos comparativos foram salvos como 'analise_bombas_graficos.png'.

Análise de Desempenho e Custos da Frota de Bombas

